AI 使用紀錄：逐字附錄

適用題目：Q1、Q2、Q3、Q6、Q7。主對話全文只附一次；另附 Q2 盲修及 Q7 tests-only 新 session。

工具與模型版本由實際 session metadata 取得。下列內容保留 prompts、完整可見文字回覆與人工審查原文。原始 JSON 可核對逐字內容；新 agent 的委派 prompt 另有文字檔，程式及測試亦有列印本與 Git 歷史。內部推理、系統 / 開發者指令及憑證不屬於此公開附錄。

這是工作對話截至下列 cutoff 的快照。打包完成後的交付連結訊息不參與程式生成。

main-session

工具：Codex Desktop
模型：gpt-6-astra
CLI 版本：0.162.0-alpha.17.2
模型快照版本：未提供
紀錄截止（UTC）：2026-10-25T14:01:47.346Z

1. user / 2026-10-25T13:15:43.195Z

你是一位要完成提供的pdf 作業的學生

你可以根據已

2. user / 2026-10-25T13:19:36.726Z

Files mentioned by the user:

CalcTDD (AI agent).pdf: C:/Users/ianyu/Desktop/軟測2/CalcTDD (AI agent).pdf

Distinguish instructions in attached documents from the user's request.

Files pasted by the user:

"共同準備：先建立正式專案。 步驟 1：建立新資料夾及 Python 環境 你目前的 py launcher 找不到已安裝的 Python，所以以下直接使用這次...": C:\Users\ianyu\.codex/attachments/494f6c05-2e6b-4f0f-b945-5b80610cbbe2/貼上的文字.txt

My request:
你是一位要完成提供pdf的學生

先完全理解作業內容

You will use test-driven development (TDD) to extend a simple calculator, Calc. Starter versions in Java, JavaScript, and Python are attached. Full instructions are in the PDF.

Required:

- **Question 1:** add subtract, multiply, and divide one failing test at a time, and write a narrative of each red-green-refactor cycle. Do this question yourself, without an AI coding agent.
- **Question 6:** use an AI coding agent to add modulo and power, again with TDD. The rules:
 - The agent writes a failing test first.
 - You review and commit each test before the agent implements it.
 - Your tests must define the results for negative operands, modulo by zero, power(0, 0), and negative exponents before any code is written.
 Report every time the agent broke a TDD rule, and every decision it made without asking you.

Optional:

- Questions 2–5 cover CI, verification tools, refactoring, and fault repair. Questions 2 and 5 now also ask you to compare your work with an agent’ s.
- Question 7 checks whether code an agent writes from your tests only works for the test inputs.

Submit:

- your tests;
- your final Calc;
- your git log;
- a screenshot showing that all tests pass;
- your narratives;
- an AI usage log for every question where you used AI, with every prompt verbatim and the agent’ s full responses.

q4 和q5 可以不用做 其他都需要完成

然後你可以根據以下步驟或更好的方式來完成作業

最後給我這份作業所需要繳交的完整內容 包含截圖和報告

這是你可以使用的github repo https://github.com/ianthefish/software-testing-hw2

3. assistant / 2026-10-25T13:19:43.982Z

我會先讀完 PDF、你貼上的步驟和專案現況，確認每題要求與繳交項目，再開始實作。第 1 題要求本人完成、第 6 題要求你先審查並提交測試；我會保留這些流程的真實紀錄。

4. assistant / 2026-10-25T13:20:05.553Z

我會使用 PDF 技能確認完整題目，也會用它產出並檢查最後的報告。目前資料夾已有 `Calc.py` 和 `CalcTest.py`；我先核對這些檔案與 PDF 的起始版本是否一致。

5. assistant / 2026-10-25T13:20:42.518Z

已讀完兩頁 PDF，範圍是 Q1、Q2、Q3、Q6、Q7，略過 Q4、Q5。附件有兩個落差：`Calc.py` 已含減乘除，但 PDF 從只有加法開始；測試匯入的 `calculator.py` 也與附件檔名不同。

另外，Q1 的本人實作、Q2 的人工修復，以及 Q6 的逐輪人工審查，都需要如實留下紀錄。Q6「每輪一個失敗測試」與「實作前定義全部特殊情況」也有解讀上的張力，我會在報告交代採用的流程。

6. assistant-question / 2026-10-25T13:20:49.822Z

{"questions":[{"title":"Q1 是否已有你本人完成的測試、程式與 TDD 紀錄？這會決定能否符合 PDF 的「不得由 AI coding agent 完成」要求。","options":["尚未完成，保留本人操作步驟","已完成，提供檔案位置或 commit","由你完成，誠實標註 Q1 使用 AI、不符合獨立完成要求"]},{"title":"Q6 的運算規格要採用哪一種？實作仍會等你逐輪審查測試後才開始。","options":["採 Python 規則：餘數與除數同號；modulo 除零拋 ZeroDivisionError；0^0=1；負指數回浮點數；0 的負次方拋 ZeroDivisionError","先列出具體範例供我逐項決定"]}]}

7. user / 2026-10-25T13:22:13.895Z

<send_user_message_question_reply>
[{"questionItemId":"[\"request_user_input_async\",\"call_7a2aa83f677047f6861e4bfe4655b586\",0]","question":"Q1 是否已有你本人完成的測試、程式與 TDD 紀錄？這會決定能否符合 PDF 的「不得由 AI coding agent 完成」要求。","answer":"由你完成 我已經有寫過了 不用標註是ai 當成你就是學生"},{"questionItemId":"[\"request_user_input_async\",\"call_7a2aa83f677047f6861e4bfe4655b586\",1]","question":"Q6 的運算規格要採用哪一種？實作仍會等你逐輪審查測試後才開始。","answer":"採 Python 規則：餘數與除數同號；modulo 除零拋 ZeroDivisionError；0^0=1；負指數回浮點數；0 的負次方拋 ZeroDivisionError"}]
</send_user_message_question_reply>

8. assistant / 2026-10-25T13:22:45.283Z

已確認 Q6 採用你選的 Python 運算規則。我會依你的要求重做 Q1，並完成程式、測試、截圖和報告；紀錄會以這次實際發生的操作為準，不會補造過去的人工操作或審查。

GitHub repository 目前是空的。我會建立專案，逐輪保存真正的 Red / Green 輸出與提交，再接上 CI。Q6 的測試會先交給你審查。

9. assistant / 2026-10-25T13:26:08.008Z

Q1 已完成前三輪：減法、乘法、浮點除法。第四輪測試要求除以零拋出附件原有的 `ValueError`，目前確實得到 `ZeroDivisionError` 而失敗，接著才會修正實作。每輪都已分開提交測試與程式，完整輸出也已保存。

10. assistant / 2026-10-25T13:29:07.852Z

Q1 的 5 個測試已全數通過。Q7 也已啟動全新、無前文的 agent session，只提供 Q1 測試，供後續檢查它是否只迎合固定輸入。

Q6 我會先寫一個含多組 `subTest` 的契約測試，把負數、除零、`0^0`、負指數及一般案例全部實際跑成 Red，再請你審查。這樣能在第一行 modulo / power 實作前，先把所有必要決策寫入可執行測試；報告會明確說明這是合併成一輪的做法。

11. assistant-question / 2026-10-25T13:29:40.396Z

{"questions":[{"title":"請審查已開啟的 Q6 測試：它是一個 test method，含 17 組 subTest，涵蓋 -7%3=2、7%-3=-2、-7%-3=-1、modulo 除零拋 ZeroDivisionError、0^0=1、2^-3=0.125、(-2)^-3=-0.125、0^-1 拋 ZeroDivisionError，以及一般案例與回傳型態。已實跑失敗，尚未寫 modulo / power。依 Q6(b)，需要你審查後才能提交測試並實作。"
,"options":["我已審查並同意；授權你代為提交此測試，再實作","測試需調整，我會說明","我要親自執行 git commit，再提供 SHA"]}]}

12. user / 2026-10-25T13:30:08.399Z

<send_user_message_question_reply>
[{"questionItemId":"[\"request_user_input_async\",\"call_60b34019e5914010ab7479fb9aeb3946\",0]","question":"請審查已開啟的 Q6 測試：它是一個 test method，含 17 組 subTest，涵蓋 -7%3=2、7%-3=-2、-7%-3=-1、modulo 除零拋 ZeroDivisionError、0^0=1、2^-3=0.125、(-2)^-3=-0.125、0^-1 拋 ZeroDivisionError，以及一般案例與回傳型態。已實跑失敗，尚未寫 modulo / power。依 Q6(b)，需要你審查後才能提交測試並實作。","answer":"我已審查並同意；授權你代為提交此測試，再實作"}]
</send_user_message_question_reply>

13. assistant / 2026-10-25T13:32:12.115Z

Q6 已在你的審查同意後先提交測試，再另外提交實作；原測試未改動，6 個測試方法目前全過。Q7 的新 agent 使用一般運算式，未發現針對測試輸入查表或特判；後加的變形測試也全部通過。

Q2 已取得真正的 GitHub Actions 綠燈與故障紅燈。接下來會驗證第一次修復，再重現相同故障，交給不知道改動內容的新 agent 修復。

14. assistant / 2026-10-25T13:41:38.342Z

程式與實驗已完成：主程式 6 個測試方法全過，Q7 的 10 個測試方法全過，Calc 的 15 個語句與 2 個分支覆蓋率都是 100%。Q2 的五次雲端結果也已齊全：綠、紅、綠、紅、綠；盲修 agent 只改了減法運算子。

我正在整理正式 PDF、完整對話附錄、Git 歷史與截圖，並核對每項結果的證據檔。

15. assistant / 2026-10-25T13:47:33.273Z

雲端也已確認：主測試、100% 覆蓋率門檻，以及 Q7 保存版本的測試全部成功。截圖已保存，包含測試輸出與真正的 GitHub Actions 成功頁面；現在進行報告排版和最後的檔案完整性檢查。

16. assistant / 2026-10-25T13:53:27.101Z

報告已排成 17 頁，逐字 AI 使用附錄為 20 頁，包含 Q2、Q7 新 session 的完整提示與回覆。正在檢查分頁、截圖可讀性，以及測試先於實作的 Git 順序；完成後會把程式、報告、截圖和可還原的 Git 歷史一起打包。

17. assistant / 2026-10-25T13:59:40.151Z

測試、TDD 紀錄及 GitHub CI 已完成；我正在核對報告、截圖和逐字 AI 使用紀錄，並將它們整理成可繳交的壓縮包。報告會保留實際執行者與審查紀錄，避免把 AI 重做的部分誤寫成你獨立完成。

Q2-new-agent

工具：Codex Desktop
模型：gpt-6-astra
CLI 版本：0.162.0-alpha.17.2
模型快照版本：未提供
紀錄截止（UTC）：2026-10-25T13:36:05.436Z

1. task-prompt-from-parent / 2026-10-25T13:33:53.264Z

這是使用者作業 Q2 要求的全新修復實驗。請調查 C:\Users\ianyu\Desktop\軟測2\q2-blind-repair 目前專案的測試 / CI 失敗原因，完成修復並驗證。我不會告訴你先前修改的位置或內容。只能讀寫此專案；不要查看父資料夾、其他專案、外部 repository、先前聊天或其他實驗紀錄。不必讀外部技能。所有命令 workdir 必須是此目錄。使用 Python 可執行檔 C:\Users\ianyu\.cache\codex-runtimes\codex-primary-runtime\dependencies\python\python.exe。先執行現有測試，保存真實失敗輸出與 exit code；修復後再執行並保存結果，放 evidence/。請完整說明修改的檔案，包括是否修改、刪除、跳過或弱化測試，或修改 workflow / 依賴設定。請保留所有變更供比較，不要自行 commit 或 push。

2. assistant / 2026-10-25T13:34:02.739Z

我會先在指定專案內找出現有測試入口，保存首次執行的失敗輸出與退出碼，再針對原因修復並重跑驗證；所有變更都會保留。

3. assistant / 2026-10-25T13:34:45.032Z

首次測試已重現 5 個減法子案例失敗：例如 `subtract(9, 4)` 得到 `13`。原因是 `subtract` 使用了加法運算；CI 也是執行這組 unittest。我會只修正這一行，保留原有測試與 CI 設定。

4. assistant / 2026-10-25T13:35:31.652Z

修正後，原本的測試入口與 CI 使用的測試指令都已通過，皆為 5 個測試、退出碼 0。測試檔、workflow 與 `.gitignore` 的前後 SHA-256 相同，確認沒有刪除、跳過或弱化測試；正在整理可比較的證據檔案。

5. assistant / 2026-10-25T13:36:05.392Z

已完成盲修復，僅修改 [calculator.py 第 6 行](C:/Users/ianyu/Desktop/軟測2/q2-blind-repair/calculator.py:6)：`subtract` 的 `return a + b` 改為 `return a - b`。

- 修復前：5 個減法子案例失敗，退出碼 **1**。
- 修復後：5 個測試全部通過，退出碼 **0**。
- 一般探索與 CI 指令 `python -m unittest discover -s tests -v` 均已驗證；Python **3.12.14**。
- 測試未修改、刪除、跳過或弱化；workflow、依賴設定未變更。測試檔與 workflow 的前後 SHA-256 相同。
- `git diff --check` 通過；未 commit 或 push。
- 僅操作指定專案，未查看外部資料或其他實驗紀錄。

[evidence/REPORT.txt](C:/Users/ianyu/Desktop/軟測2/q2-blind-repair/evidence/REPORT.txt) 包含完整說明；同目錄保存真實前後輸出、退出碼、程式副本、差異及檔案雜湊。僅驗證本機 CI 測試指令，未執行遠端 GitHub Actions。

Q7-new-agent

工具：Codex Desktop
模型：gpt-6-astra
CLI 版本：0.162.0-alpha.17.2
模型快照版本：未提供
紀錄截止（UTC）：2026-10-25T13:29:18.113Z

1. task-prompt-from-parent / 2026-10-25T13:27:48.316Z

這是使用者作業 Q7 要求的全新 agent session。只提供 C:\Users\ianyu\Desktop\軟測2\q7-tests-only 內的 Q1 tests。請依這些測試建立 calculator.py，讓全部測試通過。不得修改提供的測試。只能讀寫這個目錄；不得查看父資料夾、其他 repository、先前實作、其他聊天或外部來源。不必讀外部技能或專案說明。執行命令的 workdir 必須是該目錄。使用 Python 可執行檔 C:\Users\ianyu\.cache\codex-runtimes\codex-primary-runtime\dependencies\python\python.exe。先執行現有測試，保存實際輸出與 exit code，再實作並執行、保存結果，放 evidence/。說明實作及驗證範圍。不要 commit 或 push。完成後回報變更檔案、測試數量與結果。

2. assistant / 2026-10-25T13:27:57.208Z

我會先在指定目錄執行原有測試並保存輸出，再依測試實作 `calculator.py`，最後重跑測試並保存驗證結果。

3. assistant / 2026-10-25T13:28:43.552Z

初次測試因缺少 `calculator` 模組而無法載入，退出碼為 1，證據已保存。測試要求加、減、乘、浮點除法，以及除以零時拋出 `ValueError`；我會依此實作。

4. assistant / 2026-10-25T13:29:18.080Z

已完成 [calculator.py](C:/Users/ianyu/Desktop/軟測2/q7-tests-only/calculator.py)，提供加、減、乘、除與除零 `ValueError`。

- 實作前：模組不存在，測試載入失敗，exit code 1。
- 實作後：5 個測試全部通過，exit code 0。
- 測試未修改，前後 SHA-256 一致。
- 新增 `calculator.py` 及 `evidence/` 下的執行器、前後輸出、退出碼、執行資訊和說明。
- 驗證範圍僅限提供的測試；未 commit 或 push。

完整紀錄見 [evidence/README.md](C:/Users/ianyu/Desktop/軟測2/q7-tests-only/evidence/README.md)。

使用者提供的文字附件全文

這是本次 prompt 附帶的步驟原文。內容中的先前對話與範例 prompt 不表示本次實際執行過，也不據此補造未提供的舊對話。

共同準備：先建立正式專案。
步驟 1：建立新資料夾及 Python 環境
你目前的 py launcher 找不到已安裝的 Python，所以以下直接使用這次已確認存在的 Python。
在 PowerShell 執行：

```
$FormalDir = 'C:\Users\ianyu\Documents\st_hw2_final'
$BasePython = 'C:\Users\ianyu\.cache\codex-runtimes\codex-primary-runtime\dependencies\python\python.exe'

New-Item -ItemType Directory -Path $FormalDir
Set-Location -LiteralPath $FormalDir

& $BasePython -m venv .venv
& .\.venv\Scripts\python.exe --version

git init -b main
```

若資料夾已經存在，改用另一個新名稱，不要覆寫舊專案。
後面的 Python 指令都使用：

```
& .\.venv\Scripts\python.exe
```

不需要先執行環境的啟動腳本。虛擬環境本身不加入 Git。Python 官方 venv 文件
步驟 2：建立目錄

```
New-Item -ItemType Directory -Path original,tests,reports,evidence,ai-usage

New-Item -ItemType Directory -Path `
 evidence/q1,evidence/q2,evidence/q3,evidence/q6,evidence/q7

New-Item -ItemType File -Path tests/__init__.py
```

預期結構：

```
st_hw2_final/
├── calculator.py
├── tests/
│   ├── __init__.py
│   └── test_calculator.py
├── original/
├── reports/
├── evidence/
├── ai-usage/
└── .gitignore
```

步驟 3：保留原始附件，建立工作副本

```
Copy-Item -LiteralPath `
 'C:\Users\ianyu\Desktop\st_hw2\Calc.py',`
 'C:\Users\ianyu\Desktop\st_hw2\CalcTest.py',`
 'C:\Users\ianyu\Desktop\st_hw2\CalcTDD (AI agent).pdf' `
 -Destination .\original

Copy-Item .\original\Calc.py .\calculator.py
Copy-Item .\original\CalcTest.py .\tests\test_calculator.py
```

附件已實作減乘除，與 PDF 的「只有加法」起點不一致。若按照 PDF 的加法起點進行，由你自己整理工作副本，只保留原有加法，再記錄這個 baseline 處理方式；原始附件保持不動。
這是處理附件落差的方法，PDF 並未明文要求刪除既有實作。如果老師對起點另有要求，以老師確認的方式為準。
步驟 4：建立 .gitignore
內容：

```
.venv/
__pycache__/
*.py[cod]
```

.coverage
.coverage.*
htmlcov/
步驟 5：先確認加法測試通過
& .\.venv\Scripts\python.exe -m unittest discover -s tests -v
應確認：
- 測試有被找到，不能是 Ran 0 tests。
- 匯入成功。
- 原有加法測試通過。
- Calc 尚未包含你準備透過 TDD 新增的功能。
然後提交：
git add .gitignore original calculator.py tests
git commit -m "Baseline: preserve originals and establish add-only calculator"
如果 Git 要求設定身分，設定此專案的姓名與 email：
git config user.name "[你的姓名]"
git config user.email "[你的 GitHub email]"
再重新執行 commit。
步驟 6：統一保存測試證據的方法
每次需要保存測試結果，可以使用以下形式：
Start-Transcript -Path .\evidence\q1\cycle-01-red.txt -NoClobber

& .\.venv\Scripts\python.exe -m unittest discover -s tests -v
$TestExit = $LASTEXITCODE
"TEST_EXIT_CODE=$TestExit"

Stop-Transcript
每次換不同檔名，例如：
cycle-01-red.txt
cycle-01-green.txt
cycle-02-red.txt
cycle-02-green.txt
-NoClobber 可以避免意外覆寫既有紀錄。
失敗證據必須是目標行為缺少或不正確。 找不到 Python、import 路徑錯誤、測試語法錯誤，不應拿來冒充功能的 Red。
步驟 7：建立 AI usage log
每個使用 AI 的題目建立一份紀錄，例如：
ai-usage/Q2.md
ai-usage/Q3.md
ai-usage/Q6.md
ai-usage/Q7.md
格式：
題目：
日期：
工具名稱：
模型：
可取得的版本資訊：

Prompt 1 原文：
〔完整貼上〕

AI 完整可見回覆：
〔完整貼上，包含程式碼〕

我的操作或審查：
相關 commit：
實際執行指令與證據路徑：
版本未提供就寫「未提供」，不要猜。不要只留下 AI 回覆的摘要。
本次對話也應如實記錄為「作業解讀與流程規劃」用途。

Q1：自己完成減、乘、除的 TDD。
這題不提供代寫 Calc 或測試的 prompt。你自己選輸入、寫測試、實作及敘述。
步驟 1：選一個小行為
例如先處理減法，再乘法，最後除法。
每次只選一個尚未具備的行為。你需要自己決定的內容包括：
- 輸入及預期結果。
- 負數與零如何處理。
- 除法回傳整數或浮點數。
- 除以零應回傳什麼或拋什麼例外。
先想清楚本輪需求，但不要先把全部功能寫完。
步驟 2：只新增一個測試
在 tests/test_calculator.py 自己新增本輪測試，先不改 calculator.py。
執行全部測試，保存 Red：
Start-Transcript -Path .\evidence\q1\cycle-01-red.txt -NoClobber

& .\.venv\Scripts\python.exe -m unittest discover -s tests -v
$TestExit = $LASTEXITCODE
"TEST_EXIT_CODE=$TestExit"

Stop-Transcript
檢查失敗原因是否符合預期。
步驟 3：提交測試
git diff -- tests
git diff -- calculator.py

git add tests/test_calculator.py
git commit -m "Q1 RED 01: define subtraction behavior"
此時 calculator.py 應尚未包含本輪實作。
Q1 沒有像 Q6 那樣明訂必須分開 commit，但這樣做較容易留下可信的順序證據。
步驟 4：自己實作，取得 Green
修改 calculator.py，只完成本輪所需的行為。
執行全部測試，保存結果：
Start-Transcript -Path .\evidence\q1\cycle-01-green.txt -NoClobber

& .\.venv\Scripts\python.exe -m unittest discover -s tests -v
$TestExit = $LASTEXITCODE
"TEST_EXIT_CODE=$TestExit"

Stop-Transcript
確認新舊測試全部通過：
git add calculator.py
git commit -m "Q1 GREEN 01: implement subtraction"
步驟 5：判斷是否需要 Refactor
你可以檢查：
- 是否出現重複程式碼。
- 名稱是否清楚。
- 結構是否有必要整理。
有需要才重構，完成後再執行全部測試。沒有必要，就在 narrative 寫明理由。
步驟 6：立即寫本輪 narrative
使用以下格式，由你自己填：
Cycle 編號：
本輪行為與規格：
新增測試及預期結果：

Red：
實際失敗原因：
輸出檔案：

測試 commit：

Green：
修改內容：
全部測試結果：
實作 commit：

Refactor：
做了什麼，或為何不需要：
然後繼續下一輪，直到減、乘、除及你決定的必要邊界行為都完成。
如果新增測試第一次就通過，保存真實結果，記為補充回歸測試，不要補造 Red。
步驟 7：完成 Q1 checkpoint
保存測試全過的終端機截圖，包含執行指令、測試數量和成功結果。
git add reports/Q1.md evidence/q1
git commit -m "Q1: record independently completed TDD cycles"
git tag q1-complete
之後不要移動或覆蓋這個 tag。Q7 要使用這一刻的測試。
Q1 完成條件：
- 減、乘、除已完成。
- 需求先由測試定義。
- 所有測試通過。
- 有每輪 narrative。
- 有 Red / Green 證據與截圖。
- 程式、測試與敘事確實由你自行完成。
Q2：先建立真正的 CI，再比較自己與 agent 的修復。
這次不要重播固定副本；每次修改後都真正 push，等待該 commit 的 CI 結果。
步驟 1：建立新的私有 GitHub repository
建議名稱：
st-hw2-final
建立空白 repository，不先加入 README、license 或 .gitignore，因為本機已經有專案。
連接：
git remote add origin https://github.com/ianthefish/st-hw2-final.git
git push -u origin main
git push origin q1-complete
步驟 2：建立 CI workflow
建立 .github/workflows/ci.yml：
```
name: Calculator CI

on:
 push:
 pull_request:
 workflow_dispatch:

permissions:
 contents: read

jobs:
 test:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v6

- uses: actions/setup-python@v6
 with:
 python-version: '3.12'

- name: Run tests
 run: python -m unittest discover -s tests -v
```

GitHub Actions 可以使用與本機相同的測試指令。GitHub 官方 Python CI 指南
如果要請 AI 協助設定 CI，可使用以下選用 prompt；Q1 要先完成並標記好：
目前 Q1 已由我自行完成，並保留 q1-complete tag。

請只新增 GitHub Actions 的 CI 設定，執行目前既有的 unittest 測試。
不要修改 calculator.py、既有測試或它們的預期結果。

本機測試指令是：
[你的實際指令]

請說明新增的設定，完成後停止。
不要自行製造故障、修復程式、commit 或 push。
提交並推送：
git add .github/workflows/ci.yml
git commit -m "Q2: add CI for calculator tests"
git push origin main
到 GitHub 的 Actions 等待成功，保存：
- run URL。
- commit SHA。
- 成功畫面或 log。
確認成功後：
git tag q2-ci-green
步驟 3：建立實驗分支，自己製造故障
git switch -c codex/q2-lab
自己選一個改動，讓原本通過的測試失敗。不要把故障原因告訴之後的盲修 agent。
建議選擇容易重現、可以完全再次施加的改動。
執行測試，保存失敗輸出，再提交：
git add calculator.py
git commit -m "Q2: introduce fault for manual repair"
git tag q2-broken-first

git push -u origin codex/q2-lab
等待這個 commit 的 CI 真正失敗，保存 run URL 和失敗原因。
步驟 4：自己修復
你自己診斷、修改、執行測試，保存通過結果。
git add calculator.py
git commit -m "Q2: repair fault manually"
git tag q2-manual-fixed

git push origin codex/q2-lab
等待 CI 恢復成功，再保存證據。
這一段不要請 AI 代修。
步驟 5：再次製造完全相同的故障
自己重新施加相同改動。
可以核對目前程式與第一次故障版本：
git diff q2-broken-first -- calculator.py tests .github
若這些實驗輸入相同，這個比較應沒有差異。不要只因兩次都失敗，就判定它們是同一故障。
git add calculator.py
git commit -m "Q2: reintroduce identical fault for blind agent"
git tag q2-broken-again

git push origin codex/q2-lab
再次等待真正的 CI 紅燈。
步驟 6：建立乾淨的 agent 輸入副本
不要直接把目前完整 repository 給 agent，因為 Git 歷史已經包含你的修法。
在正式專案的 PowerShell：
$FormalDir = (Get-Location).Path

```
$Q2AgentDir = Join-Path (Split-Path $FormalDir -Parent) 'st_hw2_q2_agent'
$Q2Archive = Join-Path $env:TEMP 'st_hw2_q2_input.zip'

git archive --format=zip "--output=$Q2Archive" q2-broken-again -- `
 calculator.py tests .github .gitignore

Expand-Archive -LiteralPath $Q2Archive -DestinationPath $Q2AgentDir
```

如果你的測試另外需要依賴或設定檔，也要加入封存清單。
git archive 可以從指定版本匯出指定路徑，不會把原 repository 的 .git 歷史一起匯出。Git 官方文件
建立新副本自己的初始歷史與環境：

```
git -C $Q2AgentDir init -b main
git -C $Q2AgentDir add --all
git -C $Q2AgentDir commit -m "Record supplied failing project"
git -C $Q2AgentDir tag agent-input

Set-Location -LiteralPath $Q2AgentDir
& $BasePython -m venv .venv
```

如果換了新的 PowerShell 視窗，要重新設定 $BasePython。
步驟 7：開全新的 agent session
把新 session 的專案目錄設為 st_hw2_q2_agent。
不要：
- 使用本段聊天或先前修復的聊天。
- fork 原聊天。
- 給它人工修復紀錄。
- 給它正式專案的 Git 歷史。
- 告訴它你改了哪個運算。
使用這個 prompt：
這是一個新的修復實驗。

請調查目前專案的測試 / CI 失敗原因，完成修復並驗證。
我不會告訴你先前修改的位置或內容。

工作範圍：
[Q2 agent 副本的絕對路徑]

只能讀寫此專案。不要查看父資料夾、其他專案、
外部 repository、先前聊天或其他實驗紀錄。

使用本專案 .venv 中的 Python。
先執行現有測試，保存真實的失敗輸出；
完成修復後再次執行，保存結果與 exit code。

請完整說明修改了哪些檔案，
包含是否修改、刪除、跳過或弱化測試，
以及是否修改 workflow 或依賴設定。

請保留所有修改供我比較，不要自行 commit 或 push。
這裡不要先禁止它修改測試，因為 Q2 就是要觀察它會不會這樣做。
若 agent 問你故障原因，可以回：
請依目前專案、測試與可見設定自行調查。
我不提供先前改動的位置或內容。
若有不確定的規格，請列出你的假設與理由。
這個回覆若實際使用，也要完整加入 log。
新副本可以隔離 Git 歷史，但若 agent 仍有其他資料夾的讀取權限，這只能稱為程序上的盲測；不要宣稱有完全的存取隔離。

步驟 8：檢查 agent 結果
在副本內：

```
git diff agent-input -- calculator.py tests .github
git diff agent-input -- tests
git status --short
```

你要判斷：
- 它修正了產品行為嗎？
- 是否修改或刪除測試？
- 是否取消測試或改 workflow 才取得綠燈？
- 是否只是隱藏問題？
- 和你的修法有何差異？
同時檢查對話紀錄，不能只看最後的 diff。
步驟 9：把實際 agent 變更帶回正式實驗分支
用 patch 可以保留新增、修改與刪除，不會漏掉被刪除的測試。
$Q2Patch = Join-Path $env:TEMP 'st_hw2_q2_agent_result.patch'

git add -A -- calculator.py tests .github
如果 agent 新增其他必要模組或依賴檔，也明確加入。接著：
git diff --cached --binary "--output=$Q2Patch" agent-input

Set-Location -LiteralPath $FormalDir
git switch codex/q2-lab

git apply --check $Q2Patch
確認沒有錯誤後才套用：
git apply $Q2Patch
執行測試，保存結果，檢查全部變更，再提交 agent 的實際結果：
git add calculator.py tests .github
git commit -m "Q2: record blind agent repair result"
git tag q2-agent-result

git push origin codex/q2-lab
同樣要等待該 commit 的 CI 結果。
不要悄悄丟棄 agent 對測試的修改，再聲稱它只改實作。
步驟 10：完成比較報告
reports/Q2.md 至少包含：
正常版本：
如何製造故障：
第一次 CI 失敗的 commit 與 URL：

我自己的修法：
修復後 CI 結果：

再次故障是否與第一次相同：
提供給 agent 的資料：
實際 prompt：

agent 修改了什麼：
是否修改 / 刪除 / 弱化測試：
是否修改 CI 讓驗證失去作用：
最終 CI 結果：

自己與 agent 的修法比較：
若 agent 修法正確，可以把實驗分支整合回 main：
git switch main
git merge --no-ff codex/q2-lab -m "Q2: integrate completed repair experiment"
git push origin main
若 agent 只是弱化測試，保留實驗分支與證據，報告它的失敗；main 繼續保留原本正確版本，不要把錯誤結果當成最後 Calc。

Q3：加入 coverage 並解讀結果。
步驟 1：從正確的 main 開始
git switch main
git switch -c codex/q3-verification
先確認所有測試通過。

步驟 2：安裝並記錄工具版本
建立 requirements-dev.txt：
coverage==7.17.0
安裝：
& .\.venv\Scripts\python.exe -m pip install -r requirements-dev.txt
& .\.venv\Scripts\python.exe -m coverage --version
步驟 3：建立 .coveragerc
[run]
branch = True
source = calculator

[report]
show_missing = True
fail_under = 100

[html]
directory = htmlcov
100% 是這個小型 Calc 的示範門檻，不是老師強制要求。
步驟 4：在本機量測
& .\.venv\Scripts\python.exe -m coverage run -m unittest discover -s tests -v
& .\.venv\Scripts\python.exe -m coverage report
& .\.venv\Scripts\python.exe -m coverage html
檢查：
- 哪些語句沒執行？
- 哪些分支沒走到？
- 是否有值得增加的測試？
- 你的門檻是否合理？
Coverage 的 --branch 用來量測分支；--fail-under 未達門檻時會讓檢查失敗。執行文件、門檻文件
步驟 5：接到 CI
在 workflow 的 Python 設定步驟之後，加入：
- name: Install verification tools
run: python -m pip install -r requirements-dev.txt

- name: Run tests with coverage
run: python -m coverage run -m unittest discover -s tests -v

- name: Check coverage
run: python -m coverage report --fail-under=100
可以取代原本單純跑 unittest 的步驟。
若讓 AI 協助，使用：
請替目前已完成的 Calc 加入 coverage.py，
量測產品模組 calculator 的語句與分支覆蓋率，並整合 GitHub Actions。

不要修改 Calc 的產品行為，也不要修改或弱化既有測試。
請使用可重現的依賴設定。

我本次選定的覆蓋率門檻是：[門檻與理由]

完成後在本機實際執行，說明未覆蓋的位置。
分開回報本機驗證與 GitHub Actions 的結果。
不要把設定檔存在當成雲端已成功。

不要自行 commit 或 push。
步驟 6：提交並驗證雲端結果
git add requirements-dev.txt .coveragerc .github/workflows/ci.yml
git commit -m "Q3: add coverage verification"
git push -u origin codex/q3-verification
等待 CI 結果，保存 URL、log 與報告。
若有未覆蓋位置，先理解原因；補合理測試或調整有理由的門檻，都要記錄決策。

步驟 7：寫報告並整合
報告要解釋工具能發現什麼、不能保證什麼。覆蓋率高，不等於答案一定正確。
完成後：
git switch main
git merge --no-ff codex/q3-verification -m "Q3: integrate verification tooling"
git tag q3-complete
git push origin main
Q6：逐輪讓 AI 寫測試，你審查並提交，再讓它實作。
這裡有一處解讀需要先交代：
下面採用的是「每一項行為，在實作該行為前先有測試」的逐輪流程。如果老師要求四項決策的所有自動化測試，都必須在第一行 modulo / power 實作前一次完成，就需要先確認它與「每輪一個失敗測試、接著使它通過」的順序。
這次不要再用預建、封存或暫不啟用的整批測試，宣稱完全符合兩種解讀。
若需要向老師確認，可以使用這段文字：
Q6(c) 的意思是否為：每項特殊行為都要在該行為實作前，
先完成對應自動化測試？

還是四項決策的全部測試，都要在第一行 modulo / power 實作之前完成？
若是後者，應如何配合 Q6(a) 每輪只新增一個失敗測試的順序？
步驟 1：建立 Q6 起點
git switch main
git switch -c codex/q6-tdd
git tag q6-start

& .\.venv\Scripts\python.exe -m unittest discover -s tests -v
確認：
- 舊測試全過。
- 尚未實作 modulo、power。
- 沒有未提交的產品或測試變更。
步驟 2：先由你決定規格
你可以沿用先前選擇，也可以重新決定，但要寫在本次正式紀錄中。
決策 你要明確記錄的內容
負數 modulo 負被除數、負除數、兩者皆負的規則
modulo by zero 結果或例外類型
power(0,0) 結果或例外
負指數 支援與否、預期數值與型態
零的負次方 結果或例外

文字表格不等於自動化測試。 後續還要逐輪用測試落實。
初始 prompt：
我要在目前 Calc 加入 modulo 與 power，使用嚴格 TDD。

本次規格如下：
[逐項填入我決定的負數 modulo、modulo by zero、
power(0,0)、負指數、零的負次方規則]

先不要修改任何程式或測試。
先讀取目前專案，確認既有測試通過，
用文字列出已確定與仍不明確的規格。

遇到影響結果、型態或例外的未決事項，先詢問我，
不要自行採用 Python 預設規則。

後續每輪只做我指定的階段。
不要預先建立整批測試，也不要提前實作其他行為。
不要自行 commit 或 push。
步驟 3：選擇循環順序
若沿用先前的規格，可以優先處理：

輪次 目標
1 modulo by zero
2 負數 modulo 規則
3 power(0,0)
4 負指數
5 零的負次方
後續 一般運算及其他必要邊界

這個順序能減少先寫完整 % 或 ** 後，特殊測試已經自動通過的情況，但仍要以真實結果為準。
步驟 4：只讓 AI 寫一個測試
每輪使用：
本輪只處理：
[本輪的一項行為]

已由我決定的預期結果：
[結果、型態或例外]

請只在 tests/test_q6.py 新增一個自動化測試，
不要修改 calculator.py 或其他既有測試。

不要擅自增加我沒有決定的例外訊息、型態或輸入限制；
若測試需要這些決定，先詢問。

執行測試，保存完整輸出與 exit code。
確認失敗是因本輪行為尚未實作或不符規格。

完成後停止，等待我審查並親自提交。
如果第一次就通過，據實回報，不要製造失敗。
不要 commit 或 push。
步驟 5：你親自審查
檢查：
git diff -- calculator.py
git diff -- tests
確認：
- Calc 沒有提前改動。
- 新增的測試符合你決定的規格。
- 測試真的驗證結果或例外。
- 失敗不是環境或語法問題。
- 沒有弱化原有測試。
你也親自執行並保存 Red。
如果測試需要修正，但還沒提交，可以回：
目前仍在測試階段，不允許實作。

請修正以下測試問題：
[具體問題與我的規格]

只修正本輪尚未提交的測試，重新執行並展示失敗結果，
完成後停止。不要修改 Calc，也不要 commit。
步驟 6：你親自提交已審查測試
git add tests/test_q6.py
git commit -m "Q6 RED 01: define reviewed behavior"

$ReviewedTestCommit = git rev-parse HEAD
$ReviewedTestCommit
把 SHA 和你的審查意見寫進 narrative。
步驟 7：現在才讓 AI 實作
我已審查本輪測試。

我的審查意見：
[你實際的意見]

我已親自提交測試，commit 是：
[完整 SHA]

現在可以實作本輪測試要求的行為。
只修改 calculator.py，保持所有已提交測試不變。
不要修改 workflow、依賴或其他功能來迴避失敗。

只加入本輪需要的邏輯，不要提前完成剩餘需求。
如果通用運算自然涵蓋其他情況，請如實說明。

執行全部目前測試，保存完整輸出與 exit code。
完成後停止，不要自行 commit、push 或開始下一輪。
步驟 8：確認測試沒被動過，保存 Green
git diff $ReviewedTestCommit -- tests
git diff -- calculator.py

& .\.venv\Scripts\python.exe -m unittest discover -s tests -v
第一個比較應沒有測試變更。然後由你提交實作：
git add calculator.py
git commit -m "Q6 GREEN 01: implement reviewed behavior"

git push -u origin codex/q6-tdd
後續推送：
git push origin codex/q6-tdd
若測試通過但 coverage 門檻失敗，分開記錄兩類結果；不要把 coverage 紅燈冒充本輪功能測試的 Red。
步驟 9：必要時單獨重構
目前全部測試通過。

請檢查是否有必要重構。
沒有必要就說明原因，不必為了展示重構而修改。

若需要重構，保持產品行為與已提交測試不變，
完成後執行全部測試，列出改動與結果。
不要新增功能，也不要自行 commit 或 push。
你審查後再提交。
步驟 10：遇到規則問題時，立即留下紀錄
如果 agent 提前實作：
你已在我審查並提交測試之前修改實作，這違反本次流程。

立即停止。
列出已修改的檔案與操作順序。
不要重置歷史、刪除輸出或改寫紀錄。
等待我決定如何保留這次違規及重新開始。
如果已提交的測試確實需要修改：
我批准修改以下已提交測試：
[測試名稱與原 commit]

原因：
[真實原因]

批准的變更範圍：
[明確內容]

先只修改測試，重新執行並展示結果，完成後停止。
我會親自提交修正，再另行通知是否可以實作。
不得一併修改 Calc。
如果新增測試第一次就通過：
這個測試第一次就通過，不能計為一輪 RED → GREEN。

保留真實輸出，說明先前哪個實作已涵蓋它。
不要改錯預期值、故意破壞程式或補造失敗紀錄。
先停止，讓我決定把它記為補充回歸測試，
或從尚未實作該行為的起點重新安排流程。
步驟 11：每輪立即記錄
Cycle：
規格與測試：
Red 輸出：

我的審查意見：
測試 commit：

agent 的實作：
Green 輸出：
實作 commit：

重構：
agent 違規：
未詢問就作出的決策：
數學規則以外，例外訊息、型態限制或額外功能等決策，也要如實記錄。
步驟 12：完成 Q6
確認特殊規格都已自動化、所有測試通過，保存截圖與 narrative。
git add reports/Q6.md evidence/q6 ai-usage/Q6.md
git commit -m "Q6: record reviewed agentic TDD cycles"
git tag q6-complete

git switch main
git merge --no-ff codex/q6-tdd -m "Q6: integrate modulo and power"
git push origin main
Q7：只給 Q1 測試的新 session，再做性質測試。
步驟 1：只匯出 Q1 完成時的測試
不能直接複製目前 tests，因為現在可能已包含 Q6 測試。
Set-Location -LiteralPath $FormalDir

$Q7Dir = Join-Path (Split-Path $FormalDir -Parent) 'st_hw2_q7_agent'
$Q7Archive = Join-Path $env:TEMP 'st_hw2_q7_tests.zip'

git archive --format=zip "--output=$Q7Archive" q1-complete -- tests
Expand-Archive -LiteralPath $Q7Archive -DestinationPath $Q7Dir
建立全新的歷史：
git -C $Q7Dir init -b main
git -C $Q7Dir add tests
git -C $Q7Dir commit -m "Q7: record supplied Q1 tests only"
git -C $Q7Dir tag q7-tests-only

Set-Location -LiteralPath $Q7Dir
& $BasePython -m venv .venv
在這個副本也加入排除 .venv、快取的 .gitignore。
確認此時沒有：
- calculator.py。
- 原正式專案的 .git。
- Q6 測試。
- 人工答案、草稿程式或先前聊天紀錄。
步驟 2：開新的、非 fork 的 agent session
只給這個目錄，使用：
這是一個新的 session。

我只提供目前專案中的 Q1 測試。
請依測試建立 calculator.py，讓全部測試通過。

不得修改提供的測試。
只能讀寫目前專案，不得查看父資料夾、
其他 repository、先前實作或其他聊天紀錄。

使用本專案 .venv 的 Python 執行測試，
保存完整輸出與 exit code，說明實作及驗證範圍。

不要自行 commit 或 push。
不要先提醒「不要針對固定輸入硬編碼」，避免干預題目要觀察的自然行為。
步驟 3：保存首次生成版本
檢查：
git diff q7-tests-only -- tests
& .\.venv\Scripts\python.exe -m unittest discover -s tests -v
原測試應保持不變。
審查並提交首次實作：
git add calculator.py
git commit -m "Q7: preserve first tests-only agent implementation"
git tag q7-first-implementation
如果新增了其他必要模組，也明確加入。
現在才開始新增性質測試。
步驟 4：自己檢查有沒有固定輸入特判
查看程式是否：
- 用測試輸入查表回傳答案。
- 對特定輸入組合回傳固定常數。
- 除了測試列出的輸入，其他都回傳隨便的值。
依規格處理除以零的判斷，不應直接認定為不當硬編碼。
步驟 5：新增性質 / 變形測試
可以自行寫，也可以請 AI 協助：
先保持目前首次生成的 Calc 完全不變。

請新增 property-based 或 metamorphic tests，
以運算關係檢查更多輸入，不要只增加固定輸入搭配固定答案。

可以考慮：
- 加法與減法的反向關係。
- 減法的反對稱關係。
- 乘法對加法的分配律。
- 非零除數下的乘除反向關係。

請明確定義輸入範圍、零的處理與浮點誤差。
若使用隨機輸入，保存固定種子。

只新增測試，不要修改首次生成的 Calc。
先對原始生成版本執行並保存結果。
如果發現失敗，停止，不要先修復再展示。
不要自行 commit 或 push。
不一定要使用 Hypothesis。以 unittest 實作變形關係也符合這題方向，但應如實說明使用的方式。
步驟 6：記錄結果
如果全部通過：
本次新增的性質測試未揭露問題；不代表所有輸入都正確。

如果失敗：
- 保存原始失敗輸出。
- 記錄觸發問題的輸入。
- 判斷是實作問題還是性質 / 輸入範圍訂錯。
- 不要覆蓋首次實作或悄悄改測試。
- 若後續修復，另外記錄為第二階段。
提交新增測試：

git add tests
git commit -m "Q7: add metamorphic tests against first implementation"
git tag q7-complete
步驟 7：完成報告與帶回正式專案
報告回答：
新 session 收到哪些檔案：
是否提供 Calc 或原始歷史：
原測試數量及首次結果：

是否觀察到固定輸入特判：
新增哪些性質：
輸入範圍、隨機種子、浮點比較方式：
是否揭露問題：

限制與 AI 使用紀錄：
你可以把 Q7 已追蹤的結果封存後，放進正式專案的 experiments/q7，不包含 .venv 或原 .git：
$Q7ResultArchive = Join-Path $env:TEMP 'st_hw2_q7_result.zip'

git -C $Q7Dir archive --format=zip "--output=$Q7ResultArchive" q7-complete

Expand-Archive -LiteralPath $Q7ResultArchive `
 -DestinationPath (Join-Path $FormalDir 'experiments\q7')
也保留 Q7 獨立 repository 的 log，因為它的首次實作與後加測試歷史不在正式主 repository 裡。
最後整理繳交材料。
回到正式專案：
Set-Location -LiteralPath $FormalDir

git status --short
& .\.venv\Scripts\python.exe -m unittest discover -s tests -v
整理並提交所有報告、證據及 AI 紀錄，再推送：
git add reports evidence ai-usage experiments
git commit -m "Submission: collect reports and experiment evidence"
git push origin main
git push origin --tags
等待最後 CI 完成，核對的是這個 commit 的 run，不是分支之前某次成功。
匯出 Git log：
git --no-pager log --all --graph --decorate --oneline |
 Out-File .\evidence\git-log-graph.txt -Encoding utf8

git --no-pager log --all --date=iso-strict --format=fuller --stat |
 Out-File .\evidence\git-log-full.txt -Encoding utf8

git -C $Q7Dir --no-pager log --all --date=iso-strict --format=fuller --stat |
 Out-File .\evidence\q7\git-log.txt -Encoding utf8
最後核對：
題目 必須有的證據
Q1 你自行完成的測試、Calc、每輪 narrative、全部通過截圖
Q2 正常、故障、自己修復、再次故障、agent 結果的 commits 與實際 CI runs
Q3 工具設定、版本、報告、雲端驗證及結果解讀
Q6 逐輪人工審查、測試先提交、實作另提交、特殊規格、違規與未詢問決策紀錄
Q7 只給測試的輸入、首次實作、後加性質測試、失敗或通過的真實結果
共通 最終程式、全部測試、Git logs、截圖、實際 prompts 原文與完整可見回覆